
Système Intelligent de Matching CV / Offres d'Emploi

SARA AKROUH
MAYSSAE EL HLAL
AYA DKIOUAK
IKHLAS IBN M'RABET
MOTEE IBN M'RABET

SCIENCE DES DONNÉES ET L'INTELLIGENCE ARTIFICIELLE

2025–2026

Tuteur(s) EMSI :

Youssra El Bouzidi

Membre(s) du Jury :

Nom Prénom

prenom.nom@emsi.ma

Nom Prénom

prenom.nom@autre.edu

Résumé

Ce rapport présente la conception et la réalisation d'un système intelligent de matching entre CVs et offres d'emploi. Ce projet s'inscrit dans le cadre d'un Projet de Fin d'Année en Science des Données et Intelligence Artificielle.

L'objectif principal de ce projet est de développer une application web capable d'automatiser la mise en correspondance entre les profils des candidats et les besoins des recruteurs. La solution proposée repose sur des techniques de traitement automatique du langage naturel, d'extraction d'information, de génération d'embeddings et de calcul de similarité cosinus.

Le système permet d'extraire automatiquement les informations importantes d'un CV, de les structurer sous forme exploitable, puis de comparer le profil du candidat avec les offres disponibles afin de proposer les meilleures correspondances.

Cette solution vise à réduire le temps de traitement des candidatures, améliorer la pertinence des résultats et faciliter la recherche d'emploi pour les candidats ainsi que la sélection des profils pour les recruteurs.

Mots-clés : Matching CV, offres d'emploi, NLP, OCR, embeddings, FastAPI, React, similarité cosinus.

Summary

This report presents the design and development of an intelligent matching system between resumes and job offers. This project is carried out as part of an end-of-year project in Data Science and Artificial Intelligence.

The main objective of this project is to develop a web application capable of automating the matching process between candidate profiles and recruiters' needs. The proposed solution is based on Natural Language Processing, information extraction, embeddings generation, and cosine similarity calculation.

The system extracts relevant information from resumes, structures it into usable data, and compares candidate profiles with available job offers in order to recommend the best matches.

This solution aims to reduce recruitment processing time, improve result relevance, and help both candidates and recruiters make better decisions.

Keywords : Resume matching, job offers, NLP, OCR, embeddings, FastAPI, React, cosine similarity.

Remerciements

Nous tenons tout d'abord à exprimer notre sincère gratitude à l'ensemble de l'équipe pédagogique qui nous a accompagnées tout au long de ce Projet de Fin d'Année.

Nous remercions particulièrement notre prof Youssra El Bouzidi pour son encadrement, ses conseils et son suivi. Son accompagnement nous a permis de mieux organiser notre travail, de clarifier nos choix techniques et d'améliorer progressivement la qualité de notre solution.

Nous remercions également l'École Marocaine des Sciences de l'Ingénieur, EMSI Tanger, pour la qualité de la formation reçue et pour les moyens pédagogiques mis à notre disposition.

Enfin, nous adressons nos remerciements à tous les membres de l'équipe pour leur collaboration, leur sérieux et leur engagement durant les différentes étapes du projet.

Table des matières

Résumé	1
Summary	2
Remerciements	3
Introduction Générale	7
1 Cadre général du projet et état de l'art	8
1.1 Contexte	8
1.2 Problématique	9
1.3 Objectifs du projet	9
1.4 Méthodologie de travail	10
1.5 Analyse des plateformes de recrutement existantes	10
1.6 Approches de matching CV / offres d'emploi	11
1.6.1 Approches basées sur les mots-clés	11
1.6.2 Approches basées sur TF-IDF	12
1.6.3 Approches basées sur les embeddings	12
1.7 Authentification par reconnaissance faciale	12
1.8 Assistants conversationnels et interfaces vocales	12
1.9 Structure du rapport	13
1.10 Conclusion	13
2 Analyse et conception	14
2.1 Introduction	14
2.2 Besoins fonctionnels	14
2.3 Besoins non fonctionnels	15
2.4 Acteurs du système	15
2.5 Architecture globale	15
2.6 Pipeline de traitement (CV → extraction → embeddings → matching → ranking)	16
2.7 Diagramme de classes	18

2.8	Diagramme de séquence de connexion	20
3	Technologies utilisées	21
3.1	Introduction	21
3.2	Python	21
3.3	OCR et extraction PDF	21
3.4	LLM — Groq API	22
3.5	NLP	23
3.6	Embeddings	23
3.7	FastAPI	23
3.8	React	23
3.9	Base de données	24
3.10	Tableau récapitulatif	25
4	Réalisation de la solution	26
4.1	Introduction	26
4.2	Module candidat	26
4.2.1	Interface d'authentification classique et inscription	26
4.2.2	Tableau de bord du candidat	27
4.2.3	Téléversement de CV et zone de traitement	27
4.2.4	Visualisation du CV structuré après traitement	27
4.2.5	Liste des recommandations de postes avec scores de matching	28
4.2.6	Profil utilisateur et gestion des informations	29
4.3	Module recruteur	30
4.3.1	Tableau de bord du recruteur	30
4.3.2	Création et publication d'offres d'emploi	30
4.3.3	Classement et classement IA des candidats par offre	31
4.4	Module d'extraction CV et Offre d'emploi	31
4.4.1	Choix du LLM	31
4.4.2	Extraction des CVs	32
4.4.3	Extraction des offres d'emploi	34
4.5	Module de matching	37
4.5.1	Comparaison des modèles d'embedding	37
4.5.2	Explication des métriques	38
4.5.3	Choix du modèle embedding	39
4.5.4	Calcul de similarité et score	40
4.6	Module Chatbot d'assistance intelligent	41
4.7	Module d'authentification et suivi par Reconnaissance Faciale	43

5	Tests et résultats	44
5.1	Introduction	44
5.2	Tests réalisés	44
5.3	Résultats attendus et obtenus	44
5.3.1	Résultat extraction de CV	45
5.3.2	Résultat extraction d'offre d'emploi	45
5.4	Évaluation à grande échelle du matching sémantique	45
5.4.1	Protocole expérimental	46
5.4.2	Résultats obtenus	46
5.4.3	Analyse par offre d'emploi	46
5.5	Limites du système	47
6	Conclusion	49
6.1	Bilan global du projet	49
6.2	Bilan personnel	49
6.3	Perspectives d'amélioration	50
6.4	Limitations	51

Introduction Générale

Le recrutement est aujourd'hui un processus essentiel pour les entreprises. Cependant, il devient de plus en plus complexe à cause du grand volume de CVs et d'offres d'emploi disponibles. Les recruteurs reçoivent souvent un nombre important de candidatures pour une seule offre, ce qui rend l'analyse manuelle longue, fatigante et parfois imprécise.

De leur côté, les candidats rencontrent aussi des difficultés pour identifier les offres qui correspondent réellement à leurs compétences, leurs expériences et leurs objectifs professionnels. Les méthodes classiques basées uniquement sur les mots-clés ne permettent pas toujours de détecter les correspondances sémantiques entre un CV et une offre d'emploi.

Dans ce contexte, l'intelligence artificielle et le traitement automatique du langage naturel représentent une solution intéressante pour automatiser l'analyse des CVs et des offres d'emploi. Grâce aux techniques d'OCR, de NLP, d'embeddings et de similarité cosinus, il devient possible de comparer automatiquement les profils des candidats avec les besoins des recruteurs.

Ce rapport présente les différentes étapes de réalisation de notre projet intitulé **Système Intelligent de Matching CV / Offres d'Emploi**. Il décrit le cadre général du projet, l'étude de l'existant, l'analyse et la conception, les technologies utilisées, la réalisation technique ainsi que les tests et les résultats obtenus.

Chapitre 1

Cadre général du projet et état de l'art

1.1 Contexte

Le marché du travail mondial connaît aujourd'hui une transformation profonde, marquée par une croissance importante du volume de CVs et d'offres d'emploi générés quotidiennement. Avec l'essor des plateformes de recrutement en ligne et la généralisation des candidatures numériques, les entreprises reçoivent désormais un grand nombre de candidatures pour un seul poste ouvert. Cette réalité rend le processus de sélection manuel long, coûteux et parfois imprécis.

Du côté des recruteurs, la lecture et l'analyse manuelle de chaque CV représentent une charge de travail considérable. Une mauvaise sélection ou un traitement trop lent des candidatures peut entraîner une perte de temps, une mauvaise allocation des ressources et un retard dans les projets internes. Du côté des candidats, la recherche d'une offre réellement adaptée au profil, aux compétences et aux objectifs professionnels peut également durer plusieurs mois.

Les méthodes classiques de mise en relation entre candidats et recruteurs présentent une limite importante : elles reposent souvent sur une simple recherche par mots-clés. Or, ces méthodes ne comprennent pas réellement le sens des informations traitées. Un candidat peut utiliser une formulation différente pour décrire une compétence, tandis qu'une offre peut employer un autre vocabulaire pour exprimer le même besoin.

C'est dans ce contexte que s'inscrit notre projet. Nous proposons de concevoir une plateforme web intelligente capable d'automatiser et d'améliorer le processus de matching entre les CVs et les offres d'emploi. La plateforme intègre également une authentification par reconnaissance faciale afin de renforcer la sécurité, ainsi qu'un assistant conversationnel accessible par écrit ou par voix pour guider les utilisateurs.

1.2 Problématique

La problématique principale de ce projet peut être formulée de la manière suivante :

Comment concevoir une plateforme intelligente capable d'automatiser et d'améliorer la mise en correspondance entre les CVs et les offres d'emploi, tout en offrant une expérience utilisateur sécurisée, accessible et interactive ?

Cette problématique se décline en plusieurs sous-problèmes :

- **Hétérogénéité des formats de CVs** : les CVs peuvent avoir des structures différentes : CV classique, CV créatif, CV scanné ou CV multilingue.
- **Limite des recherches par mots-clés** : les moteurs classiques ne comprennent pas toujours les synonymes, les reformulations et le contexte.
- **Sécurité des utilisateurs** : la plateforme manipule des données personnelles et professionnelles, ce qui nécessite une authentification sécurisée.
- **Accessibilité et accompagnement** : les candidats et recruteurs peuvent avoir besoin d'un assistant pour comprendre les résultats et utiliser correctement la plateforme.
- **Pertinence et rapidité des résultats** : le système doit fournir des correspondances pertinentes dans un temps raisonnable.

1.3 Objectifs du projet

L'objectif principal de ce projet est de développer une application web intelligente permettant d'automatiser la mise en correspondance entre les CVs des candidats et les offres d'emploi des recruteurs.

Les objectifs spécifiques du projet sont les suivants :

1. Extraire automatiquement le contenu des CVs au format PDF, qu'ils soient lisibles ou scannés.
2. Identifier les informations importantes présentes dans les CVs : compétences, formations, expériences, langues et informations de contact.
3. Structurer les informations extraites des CVs et des offres d'emploi sous forme JSON.
4. Générer des représentations vectorielles des CVs et des offres d'emploi à l'aide des embeddings.
5. Calculer un score de compatibilité entre un CV et une offre en utilisant la similarité cosinus.
6. Afficher le Top-5 des meilleures correspondances pour les candidats et pour les recruteurs.

7. Intégrer une authentification sécurisée avec reconnaissance faciale.
8. Proposer un chatbot assistant capable de guider les utilisateurs par message écrit ou vocal.
9. Développer une interface web moderne, intuitive et adaptée aux candidats et aux recruteurs.

1.4 Méthodologie de travail

Pour mener à bien ce projet, nous avons adopté une démarche progressive et itérative, organisée en plusieurs phases.

La première phase a consisté à analyser les besoins fonctionnels et non fonctionnels du système. Cette étape nous a permis d'identifier les acteurs principaux, à savoir le candidat, le recruteur et l'administrateur, ainsi que les fonctionnalités attendues.

La deuxième phase a porté sur la conception de l'architecture globale du système. Nous avons défini les différentes couches de l'application : interface utilisateur, backend, intelligence artificielle et base de données.

La troisième phase correspond au développement des modules principaux : extraction des CVs, extraction des offres d'emploi, génération des embeddings, calcul du matching, authentification par reconnaissance faciale et chatbot assistant.

La quatrième phase a été consacrée à l'intégration des modules dans une API REST, puis au développement de l'interface web. Enfin, la dernière phase a porté sur les tests et la validation du fonctionnement global de la solution.

1.5 Analyse des plateformes de recrutement existantes

Avant d'étudier les approches techniques de matching, il est important d'analyser les plateformes de recrutement existantes afin de mieux situer notre solution.

LinkedIn est l'une des plateformes professionnelles les plus utilisées. Elle propose des recommandations d'offres et de profils, mais son système de matching reste principalement basé sur des algorithmes propriétaires, sans toujours fournir un score de compatibilité clair au candidat.

Indeed est un moteur de recherche d'emploi qui regroupe des offres provenant de plusieurs sources. Il repose généralement sur des filtres classiques comme le poste, la localisation ou le type de contrat. Toutefois, la pertinence sémantique des résultats reste limitée.

Rekrute et **Emploi.ma** sont des plateformes connues au Maroc. Elles permettent aux candidats de déposer leurs CVs et aux recruteurs de publier des offres, mais elles

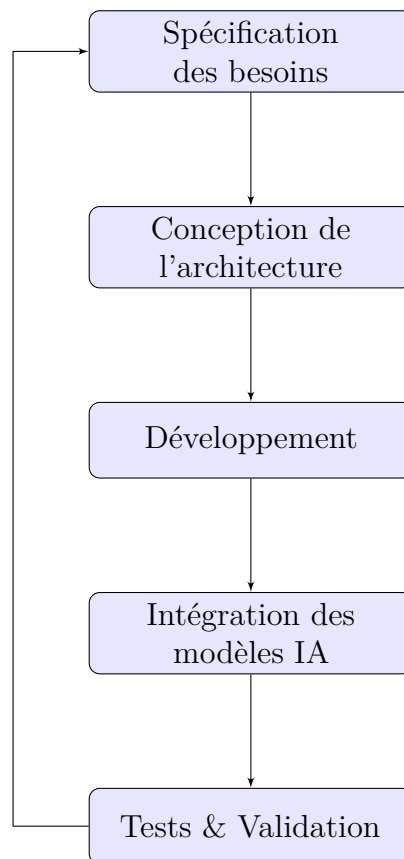


FIGURE 1.1 – Méthodologie itérative adoptée

n'intègrent pas toujours un module de matching intelligent basé sur un score explicite.

Talentsoft et **SAP SuccessFactors** sont des solutions RH avancées destinées aux grandes entreprises. Elles proposent des fonctionnalités riches, mais leur coût et leur complexité les rendent moins accessibles aux petites structures et aux candidats individuels.

Notre solution se distingue par l'intégration d'un score de compatibilité explicite, d'un matching sémantique, d'une authentification par reconnaissance faciale et d'un chatbot assistant accessible par texte ou par voix.

1.6 Approches de matching CV / offres d'emploi

1.6.1 Approches basées sur les mots-clés

Les premières approches de matching entre CVs et offres d'emploi reposent sur la comparaison des mots présents dans les deux documents. Par exemple, si une offre mentionne la compétence **Python** et que le CV contient également ce terme, le système considère qu'il existe une correspondance.

Cette méthode est simple et rapide, mais elle présente une limite importante : elle ne comprend pas le sens du texte. Elle peut donc ignorer des profils pertinents lorsque les

compétences sont exprimées avec des synonymes ou des formulations différentes.

1.6.2 Approches basées sur TF-IDF

TF-IDF, ou *Term Frequency - Inverse Document Frequency*, est une méthode statistique qui permet de représenter un document sous forme de vecteur. Elle donne plus d'importance aux mots significatifs et moins d'importance aux mots très fréquents.

Cette approche est plus avancée qu'une simple recherche par mots-clés, mais elle reste limitée, car elle ne capture pas réellement le sens des mots. Deux documents exprimant la même idée avec des termes différents peuvent être considérés comme éloignés.

1.6.3 Approches basées sur les embeddings

Les embeddings représentent une avancée importante dans le traitement automatique du langage naturel. Ils permettent de transformer les mots, phrases ou documents en vecteurs numériques capables de capturer une partie du sens du texte.

Dans notre projet, les embeddings permettent de représenter un CV et une offre d'emploi dans un même espace vectoriel. Cela permet de comparer leur proximité sémantique, même lorsqu'ils n'utilisent pas exactement les mêmes mots.

1.7 Authentification par reconnaissance faciale

L'authentification est une composante importante dans une plateforme qui manipule des données personnelles et professionnelles. Les méthodes classiques basées sur l'email et le mot de passe peuvent présenter certaines limites, notamment en cas de mot de passe faible, oublié ou compromis.

La reconnaissance faciale constitue une alternative moderne et pratique. Elle repose sur l'analyse des caractéristiques du visage afin de confirmer l'identité de l'utilisateur. Cette fonctionnalité permet d'améliorer la sécurité de la plateforme tout en simplifiant l'accès pour les candidats et les recruteurs.

1.8 Assistants conversationnels et interfaces vocales

Les assistants conversationnels jouent un rôle important dans les applications modernes. Ils permettent d'accompagner les utilisateurs, de répondre à leurs questions et de faciliter l'accès aux fonctionnalités de la plateforme.

Dans notre projet, le chatbot assistant permet aux candidats et aux recruteurs de poser des questions sur l'utilisation de la plateforme, les CVs, les offres et les scores de matching. Il peut recevoir des messages écrits ou vocaux, ce qui rend l'application plus interactive et plus accessible.

L'interface vocale repose sur deux principes : la reconnaissance vocale, qui transforme la voix de l'utilisateur en texte, et la synthèse vocale, qui permet au système de répondre oralement. Cette combinaison améliore l'expérience utilisateur et rend la plateforme plus moderne.

1.9 Structure du rapport

Ce rapport est organisé de manière à présenter progressivement les différentes étapes du projet :

1. Le **premier chapitre** présente le cadre général du projet, l'état de l'art, la problématique, les objectifs et la méthodologie de travail.
2. Le **deuxième chapitre** présente l'analyse et la conception du système : besoins fonctionnels et non fonctionnels, acteurs, architecture globale et pipeline de traitement.
3. Le **troisième chapitre** décrit les technologies et outils utilisés dans le développement de la solution.
4. Le **quatrième chapitre** présente la réalisation de la solution : modules d'extraction, de matching, d'authentification par reconnaissance faciale, chatbot, API REST et interface web.
5. Le **cinquième chapitre** présente les tests réalisés, les résultats obtenus, les limites du système et les perspectives d'amélioration.
6. Le **sixième chapitre** présente la conclusion générale du projet.

1.10 Conclusion

Ce chapitre a présenté le cadre général du projet ainsi que l'étude de l'existant liée au recrutement intelligent. Les limites des méthodes classiques, notamment la recherche par mots-clés et TF-IDF, montrent l'intérêt d'une approche plus avancée basée sur les embeddings et la similarité cosinus. L'intégration de la reconnaissance faciale et d'un chatbot assistant permet également d'améliorer la sécurité, l'accessibilité et l'expérience utilisateur de la plateforme. “

Chapitre 2

Analyse et conception

2.1 Introduction

La phase d'analyse et de conception permet de formaliser les exigences du système et de modéliser son architecture. Cette étape est cruciale pour structurer le développement et s'assurer que les objectifs métiers et techniques soient respectés.

2.2 Besoins fonctionnels

Le système doit répondre aux exigences fonctionnelles suivantes, réparties selon les profils utilisateurs :

- **Module d'authentification commun :**

- Inscription classique avec rôle (candidat/recruteur).
- Connexion par mot de passe sécurisé.
- Double authentification ou connexion rapide par reconnaissance faciale à partir d'un flux webcam.

- **Espace Candidat :**

- Téléversement de CV au format PDF.
- Visualisation du CV structuré après extraction automatique (OCR + LLM).
- Consultation des offres d'emploi recommandées avec un score de matching détaillé.
- Interaction avec le chatbot pour obtenir des conseils sur son CV ou des suggestions d'offres.
- Suivi des candidatures soumises.

- **Espace Recruteur :**

- Création et publication d'offres d'emploi avec champs structurés.

- Visualisation des meilleurs candidats pour chaque offre, triés par score sémantique.
- Graphiques d'analyse sur le flux de candidatures.
- Chatbot d'aide à la sélection (interroger la base candidat).

2.3 Besoins non fonctionnels

- **Performance** : L'extraction et la génération des embeddings d'un nouveau CV doivent s'effectuer en moins de 5 secondes. Le calcul du matching vectoriel sur les offres ne doit pas dépasser 500 millisecondes.
- **Disponibilité et Robustesse** : Le backend doit intégrer un système de secours (fallback) local basé sur des expressions régulières au cas où l'API externe de Groq ou le serveur local Ollama seraient inaccessibles.
- **Sécurité** : Les mots de passe doivent être hachés en base de données avec l'algorithme `bcrypt`. Les vecteurs biométriques faciaux ne sont pas stockés directement ; seules les images de référence sont conservées dans un espace de stockage sécurisé.
- **Responsive Design** : L'interface utilisateur doit s'adapter automatiquement aux terminaux mobiles, tablettes et ordinateurs de bureau.

2.4 Acteurs du système

Le système distingue trois principaux acteurs :

1. **Le Candidat** : Chercheur d'emploi qui téléverse son CV, consulte les offres matchées et postule.
2. **Le Recruteur** : Représentant d'entreprise qui publie des offres et consulte les profils recommandés.
3. **Le Système (ou Administrateur)** : Gère les comptes, supervise les services d'arrière-plan (Ollama, API Groq, PostgreSQL).

2.5 Architecture globale

L'architecture logicielle retenue est une architecture en couches (3-Tier) qui sépare la présentation, la logique métier et la persistance des données :

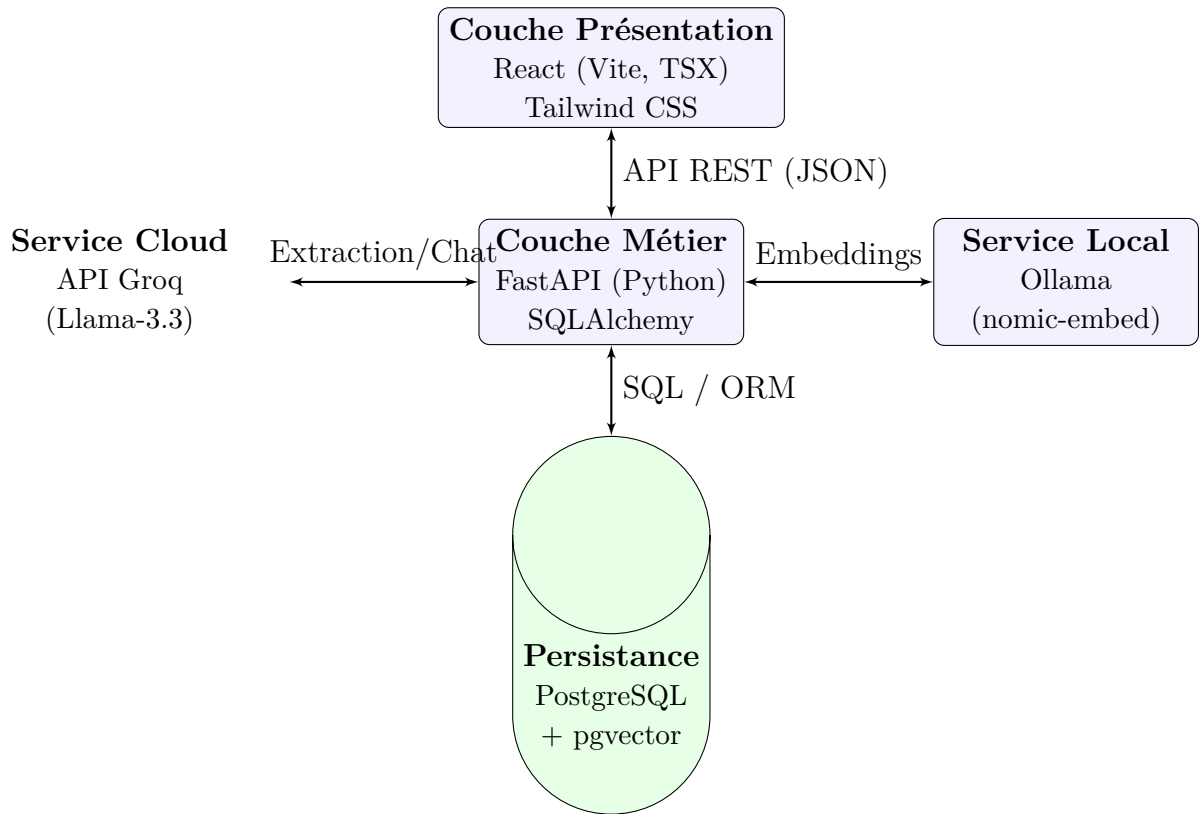


FIGURE 2.1 – Architecture globale 3-Tier de la plateforme

2.6 Pipeline de traitement (CV → extraction → embeddings → matching → ranking)

Le processus de traitement d'un CV lors de son téléversement par un candidat suit les étapes séquentielles ci-dessous :

1. **Téléversement (PDF)** : Le candidat dépose son fichier PDF sur l'interface React.
2. **Extraction Textuelle (OCR/PDF)** : Le backend FastAPI reçoit le fichier. Le module `ocr_extractor.py` utilise en priorité `pdfplumber` pour lire le texte natif. Si le texte extrait est insuffisant (< 100 caractères), il bascule sur un traitement OCR par pages avec `pytesseract` après rasterisation.
3. **Structuration Sémantique (Groq LLM)** : Le texte brut est envoyé au modèle Llama-3.3-70b-versatile avec un prompt strict pour obtenir un format JSON normalisé en anglais. En cas d'erreur de communication, un parseur regex local prend le relais.
4. **Vectorisation (Embeddings)** : Les sections structurées du CV (titre du profil, compétences, expérience, projets) sont vectorisées séparément en vecteurs de dimension 768 via le modèle de plongement `nomic-embed-text` d'Ollama ou `text-embedding-3-small` d'OpenAI.

5. **Indexation en Base** : Les données textuelles structurées sont stockées dans la table `cvs`, tandis que les vecteurs associés sont stockés dans la table `cv_embeddings` avec leur type d'embedding.
6. **Calcul du Matching Pondéré** : Lors de la recherche d'offres, une requête SQL effectue le calcul des similarités cosinus par section entre les embeddings du CV et ceux des offres d'emploi, pondère ces scores et ajoute un bonus basé sur le recouvrement exact des mots-clés de compétences.
7. **Ranking** : Les correspondances sont classées par score décroissant et renvoyées à l'utilisateur sous forme de liste paginée.

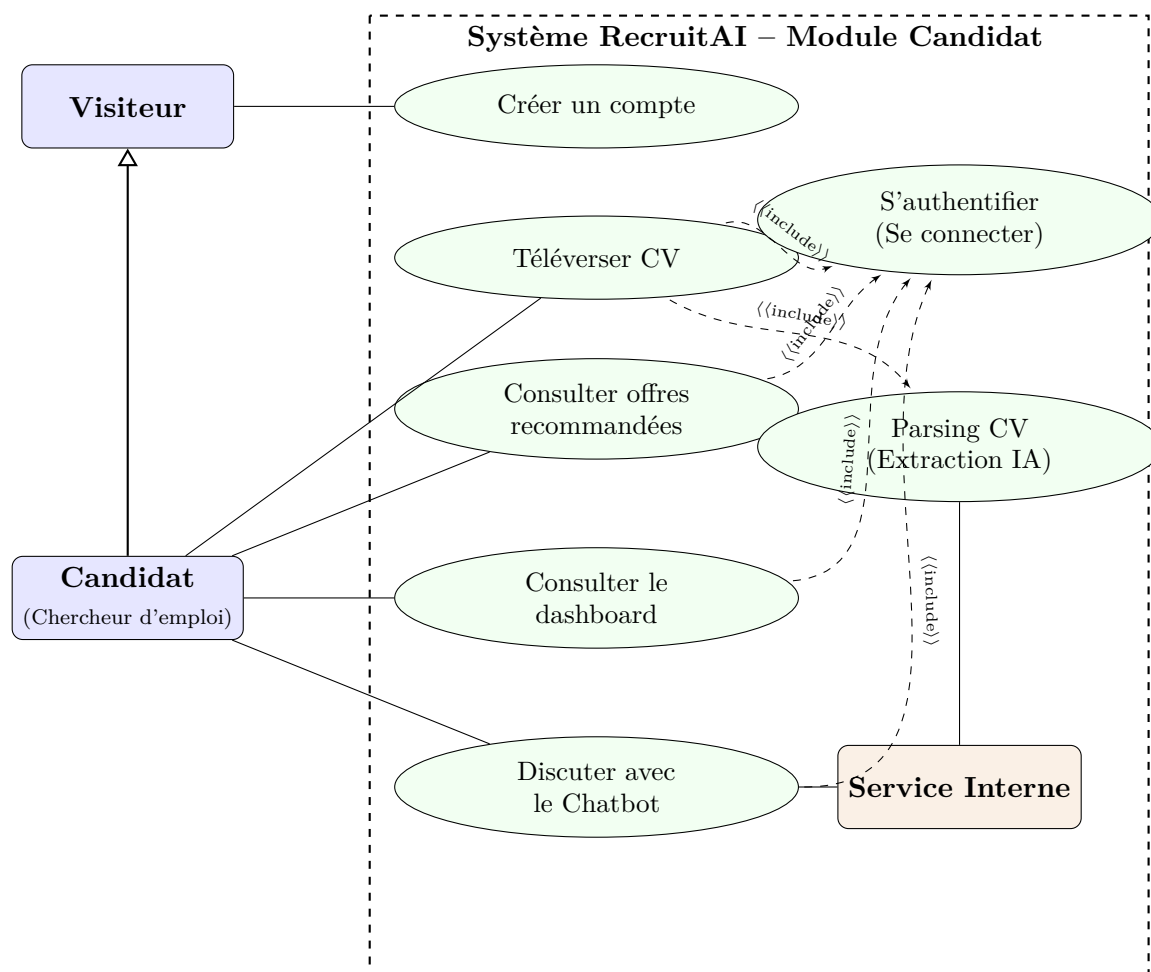


FIGURE 2.2 – Diagramme des cas d'utilisation du module Candidat et Visiteur

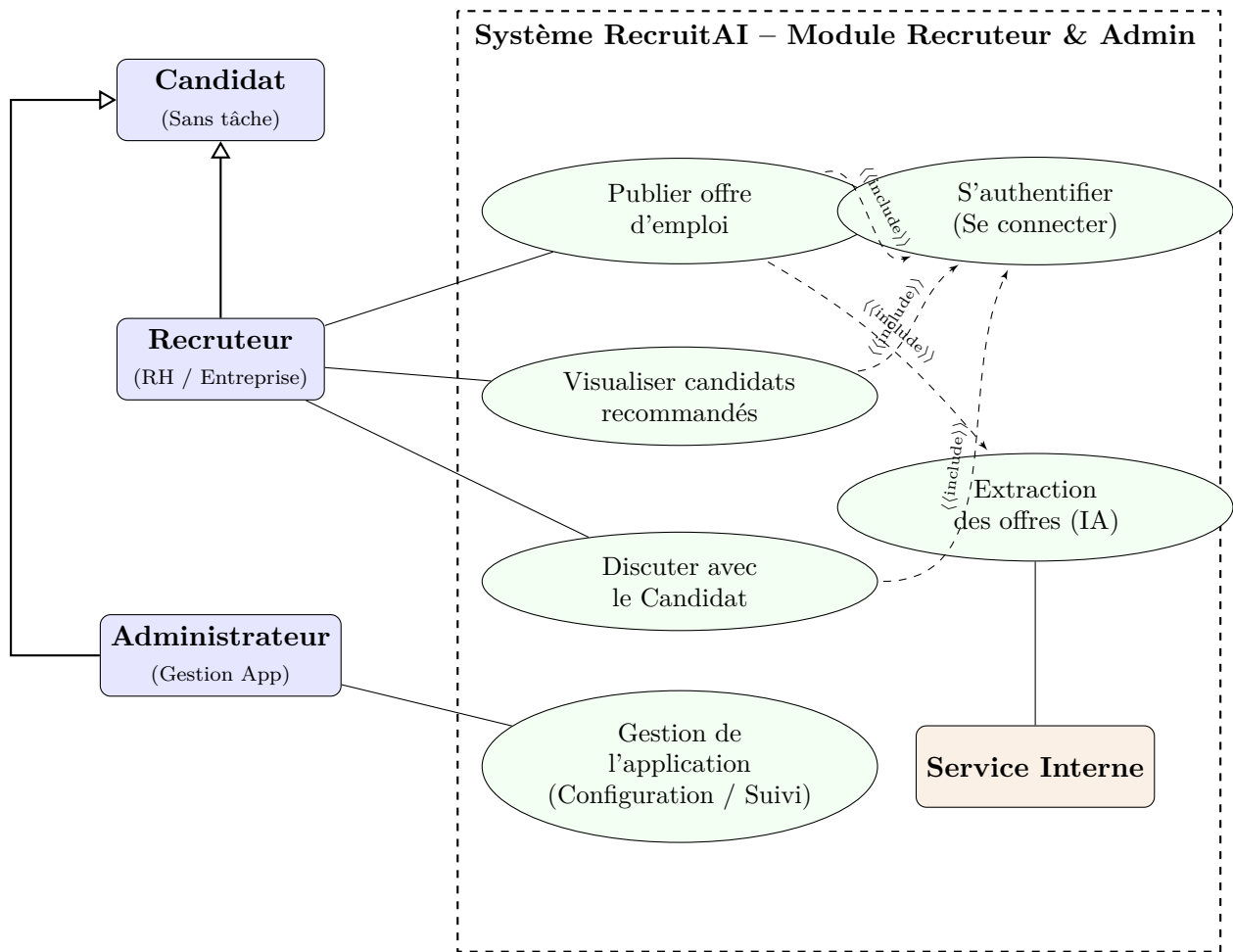


FIGURE 2.3 – Diagramme des cas d'utilisation du module Recruteur et Administrateur

2.7 Diagramme de classes

Le diagramme de classes ci-dessous modélise la structure de la base de données relationnelle et vectorielle du projet. Il met en évidence la centralisation des utilisateurs et le stockage séparé des embeddings de chunks pour les CVs et les offres.

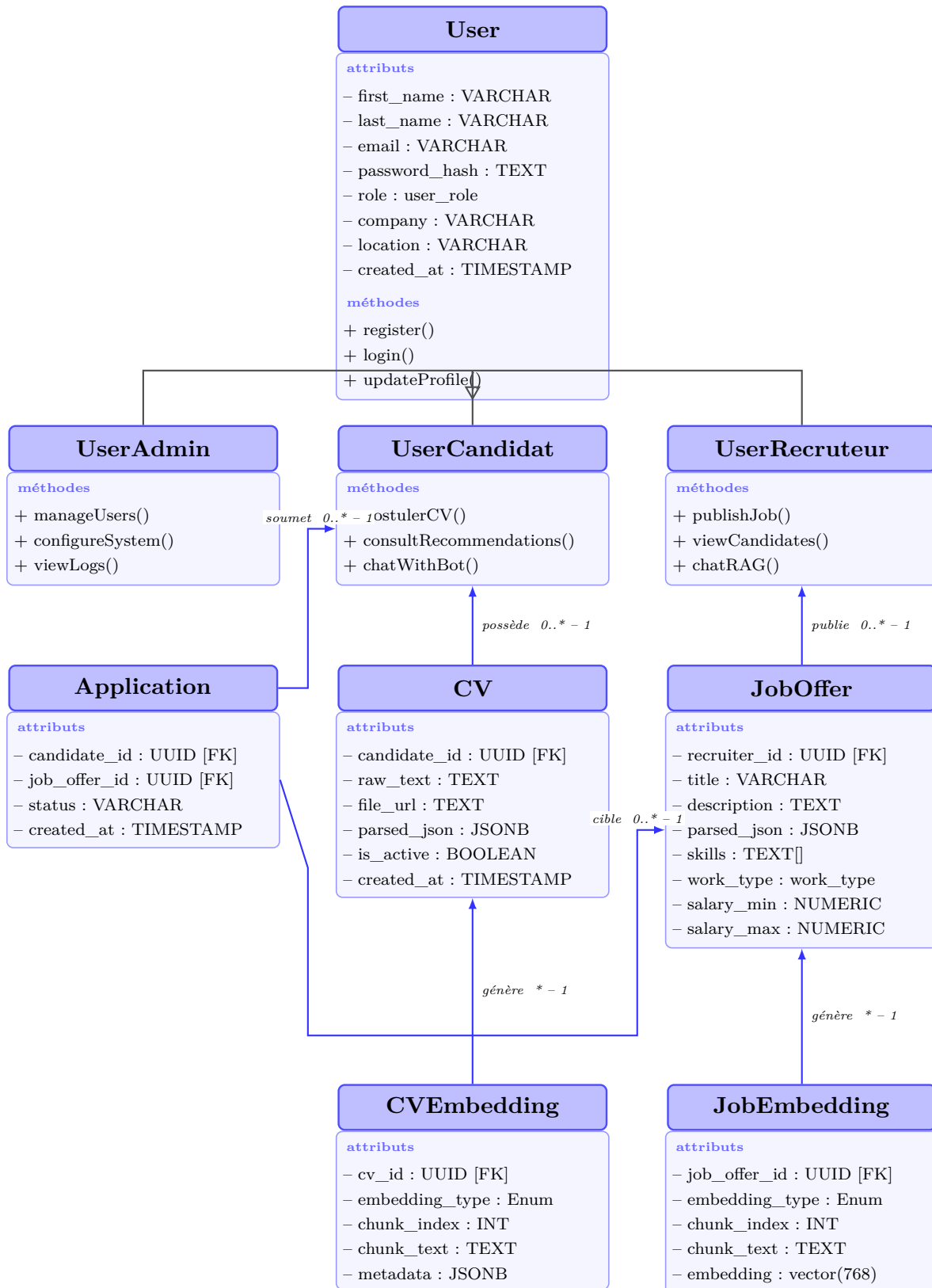


FIGURE 2.4 – Diagramme de classes UML de la base de données relationnelle et vectorielle

2.8 Diagramme de séquence de connexion

Le diagramme de séquence suivant illustre le flux d'interactions simplifiées lors d'une connexion par reconnaissance faciale.

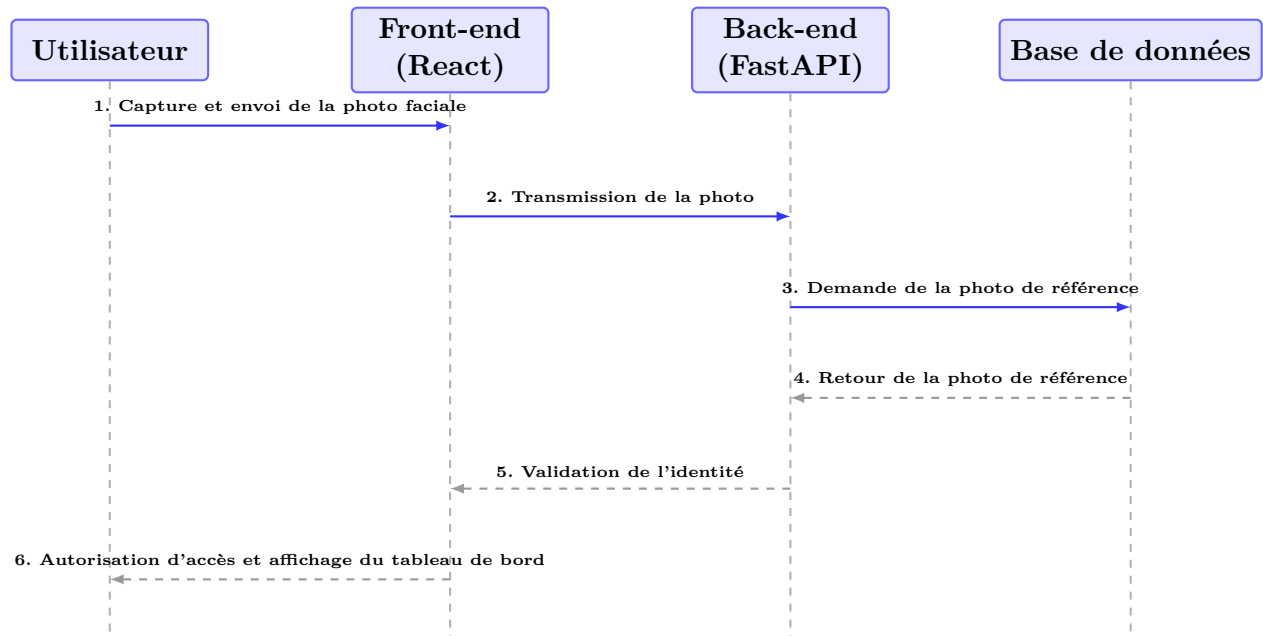


FIGURE 2.5 – Diagramme de séquence simplifié de la connexion par reconnaissance faciale

Chapitre 3

Technologies utilisées

3.1 Introduction

Ce chapitre présente les principales technologies utilisées dans le développement de notre système intelligent de matching CV / offres d'emploi. Ces technologies ont été choisies en fonction de leur performance, leur complémentarité et leur adaptation aux besoins du projet.

3.2 Python

Python est utilisé comme langage principal pour développer l'ensemble des modules du système : extraction de texte, traitement NLP, génération d'embeddings et calcul de similarité. Ce langage est largement adopté dans le domaine de la data science et de l'intelligence artificielle grâce à sa simplicité, sa lisibilité et la richesse de son écosystème de bibliothèques. Sa flexibilité permet également d'intégrer facilement des APIs externes comme Groq.



FIGURE 3.1 – Logo Python

3.3 OCR et extraction PDF

L'extraction de texte constitue la première étape du pipeline de traitement. Elle permet de récupérer le contenu textuel des CVs à partir de fichiers PDF, quelle que soit leur nature.

Trois bibliothèques complémentaires ont été utilisées :

- **pdfplumber** : bibliothèque principale pour l'extraction du texte depuis les PDFs contenant du texte sélectionnable. Elle respecte la mise en page du document, notamment pour les CVs avec des colonnes ou des tableaux.
- **PyMuPDF (fitz)** : utilisé pour convertir chaque page PDF en image haute résolution (300 DPI) lorsque le document est scanné. Cette conversion est nécessaire avant d'appliquer la reconnaissance optique de caractères.
- **Tesseract OCR** : moteur de reconnaissance optique de caractères utilisé pour lire les CVs scannés. Il analyse les images générées par PyMuPDF et en extrait le texte. Il supporte plusieurs langues dont le français et l'anglais.

Le module détecte automatiquement le type de PDF reçu : si pdfplumber extrait moins de 100 caractères, le système bascule automatiquement vers Tesseract OCR.

3.4 LLM — Groq API

Pour structurer intelligemment le texte brut extrait des CVs, nous avons utilisé l'API Groq avec le modèle **LLaMA 3.3 70B**. Ce modèle de langage de grande taille (LLM) reçoit le texte brut du CV et retourne un JSON normalisé contenant toutes les informations structurées.

L'utilisation d'un LLM présente plusieurs avantages par rapport à une approche classique basée sur des expressions régulières :

- Il comprend le sens du texte et non uniquement les mots-clés exacts.
- Il fonctionne avec n'importe quelle structure de CV.
- Il supporte plusieurs langues : français, anglais, arabe.
- Il garantit toujours la même structure JSON en sortie, avec `null` pour les champs absents.

Groq a été choisi car il offre un accès gratuit au modèle LLaMA 3.3 70B, avec une vitesse d'inférence très élevée, ce qui le rend adapté à un projet académique comme le nôtre.



FIGURE 3.2 – Logo Python

3.5 NLP

Le traitement automatique du langage naturel (NLP) permet d'analyser le texte extrait des CVs et des offres d'emploi. Il est utilisé pour détecter et identifier les informations importantes : compétences, expériences, formations, langues et autres données pertinentes. Dans notre projet, le NLP est combiné avec les embeddings pour améliorer la précision du matching sémantique entre les CVs et les offres d'emploi.

3.6 Embeddings

Les embeddings permettent de transformer les CVs et les offres d'emploi en vecteurs numériques de haute dimension. Ces vecteurs capturent le sens sémantique du texte, ce qui permet de comparer deux documents même lorsqu'ils n'utilisent pas exactement les mêmes mots. Dans notre projet, les embeddings sont générés à partir des données structurées extraites des CVs et des offres, puis utilisés pour calculer la similarité cosinus.

3.7 FastAPI

FastAPI est utilisé pour développer l'API REST du projet. Cette API joue le rôle d'intermédiaire entre l'interface web et les modules d'intelligence artificielle. Elle expose plusieurs endpoints permettant :

- l'upload des fichiers PDF de CVs ;
- l'extraction et la structuration des données ;
- le calcul du score de matching ;
- la récupération des résultats par le frontend ;
- la gestion des CVs et des offres d'emploi.



FIGURE 3.3 – Logo Python

FastAPI a été choisi pour sa rapidité, sa documentation automatique via Swagger et sa compatibilité native avec Python.

3.8 React

React est utilisé pour développer l'interface web de l'application. Il permet de créer une interface moderne, interactive et facile à utiliser. Grâce à React, l'utilisateur peut

uploader un CV, ajouter une offre d'emploi et consulter les résultats de matching sous forme de liste classée. React facilite également la communication avec l'API FastAPI via des requêtes HTTP.



FIGURE 3.4 – Logo Python

3.9 Base de données

Une base de données est utilisée pour stocker les CVs, les offres d'emploi, les données structurées extraites et les résultats de matching. Deux options ont été envisagées :

- **SQLite** : solution légère adaptée au développement et aux tests locaux, ne nécessitant aucune configuration serveur.
- **PostgreSQL** : solution plus robuste adaptée à un déploiement en production, offrant de meilleures performances pour des volumes de données importants.



FIGURE 3.5 – Logo Python

3.10 Tableau récapitulatif

Catégorie	Technologie	Version / Modèle	Rôle dans le projet
Langage Backend	Python	3.10+	Développement de l'API et
Framework Web	FastAPI	0.110+	Exposition des endpoints R
Base de Données	PostgreSQL	15+ (avec pgvector)	Stockage relationnel et calcul
Extraction PDF	pdfplumber / PyMuPDF	Dernières versions	Extraction du texte brut de
OCR	Tesseract	5.0+ (fra+eng)	Fallback pour les CVs PDF
LLM	Groq Llama	Llama-3.3-70b	Structuration des CVs et gé
Embeddings	Ollama / OpenAI	nomic-embed-text	Génération de vecteurs sém
Vision	DeepFace	0.0.80+ (VGG-Face)	Extraction des traits faciaux
Framework Frontend	React (TypeScript)	18.2+ (Vite)	Interface utilisateur dynami
Mise en page UI	Tailwind CSS	v4.0	Charte graphique, animatio

TABLE 3.1 – Écosystème technologique du projet RecruitAI

Chapitre 4

Réalisation de la solution

4.1 Introduction

Ce chapitre présente les principaux modules réalisés dans le cadre du projet. La solution est composée d'un module candidat, d'un module recruteur, d'un module d'extraction, d'un module de matching, d'une API REST et d'une interface web.

4.2 Module candidat

Le module destiné au chercheur d'emploi se concentre sur l'ergonomie, la simplicité d'utilisation et la réactivité. L'interface propose une navigation fluide pour gérer son profil, téléverser son CV, consulter ses scores de matching et interagir avec l'assistant virtuel.

4.2.1 Interface d'authentification classique et inscription

L'accès à la plateforme commence par une page de connexion et d'inscription sécurisée. L'interface utilise des formulaires dynamiques avec validation en temps réel.

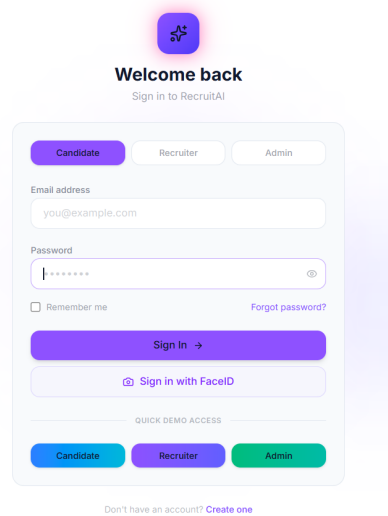


FIGURE 4.1 – Interface de connexion et d’inscription classique

4.2.2 Tableau de bord du candidat

Dès sa connexion, le candidat accède à un tableau de bord (Dashboard) centralisant les statistiques clés de son profil : le nombre de candidatures en cours, le nombre d’offres recommandées et l’adéquation globale moyenne de son profil.

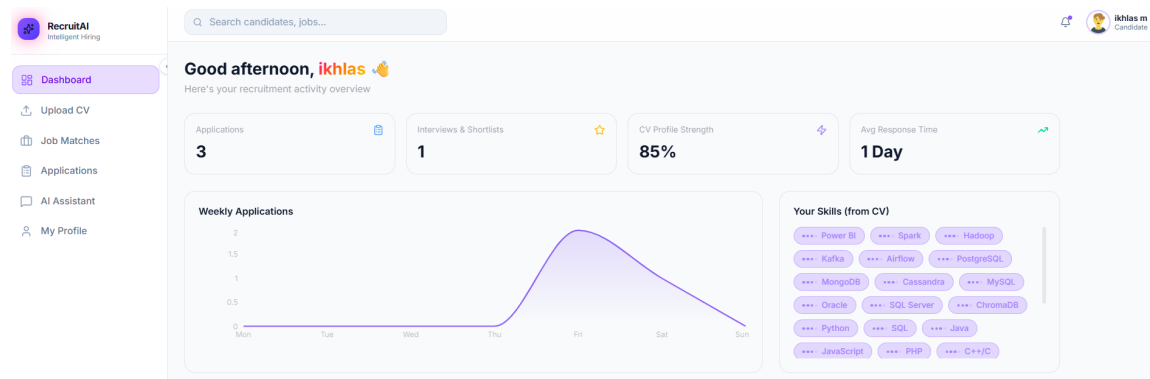


FIGURE 4.2 – Tableau de bord principal du candidate

4.2.3 Téléversement de CV et zone de traitement

L’interface d’importation de CV utilise un composant drag-and-drop moderne avec un indicateur de progression visuel pendant la phase d’extraction textuelle par le backend.

4.2.4 Visualisation du CV structuré après traitement

Après le parsing, le candidat peut visualiser sous forme de sections structurées et éditables les informations extraites par le modèle Llama-3.3-70b-versatile de Groq.

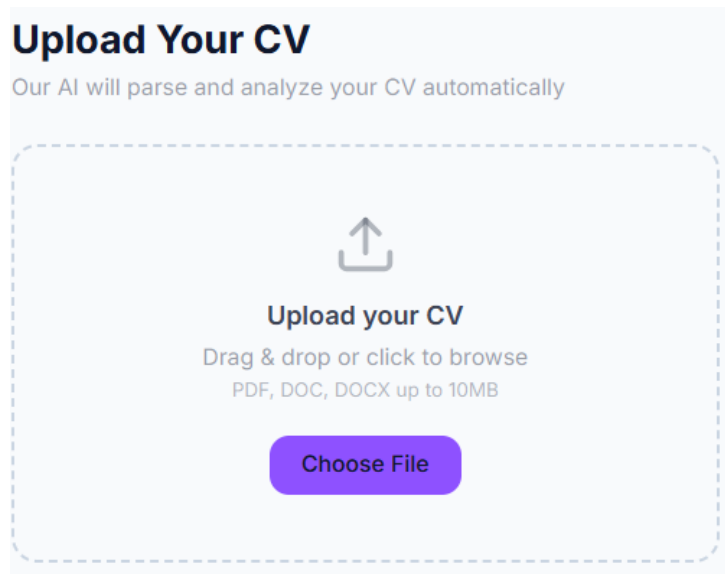


FIGURE 4.3 – Interface d'importation de CV

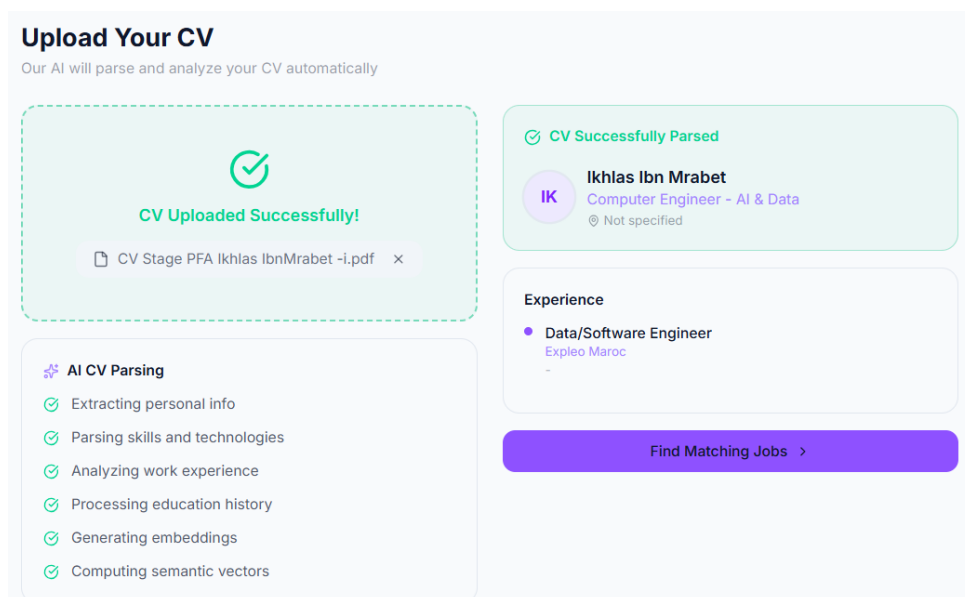


FIGURE 4.4 – Visualisation des données structurées extraites du CV

4.2.5 Liste des recommandations de postes avec scores de matching

La page **Job Matches** présente la liste des offres d'emploi classées par ordre décroissant de pertinence. Le candidat dispose d'un détail visuel du calcul du score de matching sémantique.

Job Matches

AI-powered semantic matching for your profile

Q Search jobs, companies...

Filter

Data & AI Engineer Intern

hybrid

emsi

Tangier 1-3 years 4 applicants

We are looking for a motivated Data & AI Engineer Intern to join our team. The intern will work on data engineering, machine learning, and AI projects, helping design and develop data pipelines, build AI-powered applications, and...

Python SQL Machine Learning Deep Learning Data Analysis +9

Posted Jun 21

Apply Now

Full Stack Engineer Java

remote

emsi

Toronto, Ontario, Canada 1-3 years 4 applicants

We are seeking a Full Stack Software Engineer to join our Product Development team. You will be responsible for designing, developing, and maintaining modern web applications that deliver exceptional user experiences. Working...

JavaScript TypeScript React Next.js Node.js +6

Posted Jun 21

Apply Now

Data & AI Engineer Intern

emsi

We are looking for a motivated Data & AI Engineer Intern to join our team. The intern will work on data engineering, machine learning, and AI projects, helping design and develop data pipelines, build AI-powered applications, and support model deployment. This internship offers hands-on experience with modern data and AI technologies in a collaborative environment.

REQUIREMENTS

- Currently pursuing or recently completed a degree in Computer Science, Data Science, AI, or a related field
- Knowledge of Python programming and SQL
- Understanding of machine learning fundamentals
- Experience with data analysis libraries such as Pandas and NumPy
- Familiarity with Git version control

✓ Applied

Optimize CV with AI

FIGURE 4.5 – Interface de recommandation et de matching d'offres

4.2.6 Profil utilisateur et gestion des informations

L'interface de profil permet de compléter ou modifier manuellement les informations personnelles, le numéro de téléphone, l'avatar ainsi que la photo de référence pour l'authentification faciale.

My Profile

Keep your profile updated to improve job matches

Save Changes

ikhlas m
Computer Engineer - AI & Data

Links

github.com/profile

linkedin.com/in/profile

portfolio.com

Basic Information

First Name: ikhlas

Last Name: m

Email: i@gmail.com

Phone: +212 664933640

Location: Tangier, Morocco

Company: Current Company

Professional Summary: Experienced developer with solid skills.

FIGURE 4.6 – Interface d'édition du profil du candidat

4.3 Module recruteur

Le portail recruteur privilégie une approche analytique et décisionnelle. Il permet de gérer le cycle de vie des offres d'emploi et de visualiser instantanément les meilleurs candidats pour chaque poste.

4.3.1 Tableau de bord du recruteur

La page d'accueil recruteur récapitule l'ensemble des offres d'emploi actives et affiche le nombre de candidatures reçues ainsi que le score de matching moyen pour chaque offre.

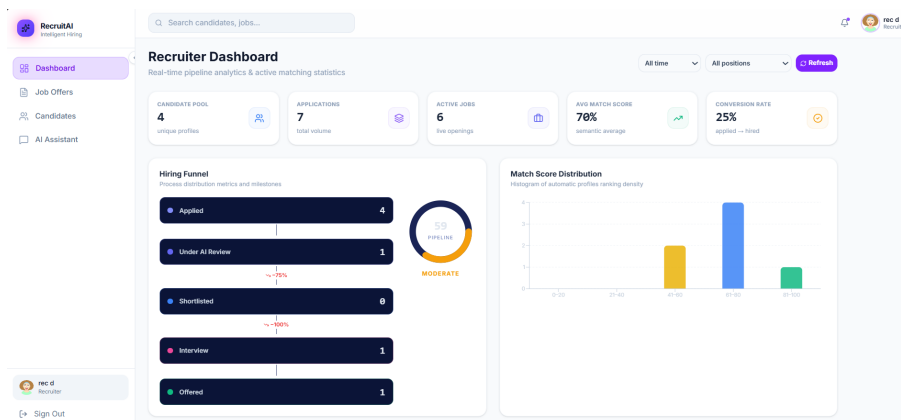


FIGURE 4.7 – Tableau de bord principal du recruteur

4.3.2 Création et publication d'offres d'emploi

Ce formulaire permet de renseigner les détails du poste. La description textuelle saisie par le recruteur est analysée pour générer des embeddings de compétences et de niveau d'expérience.

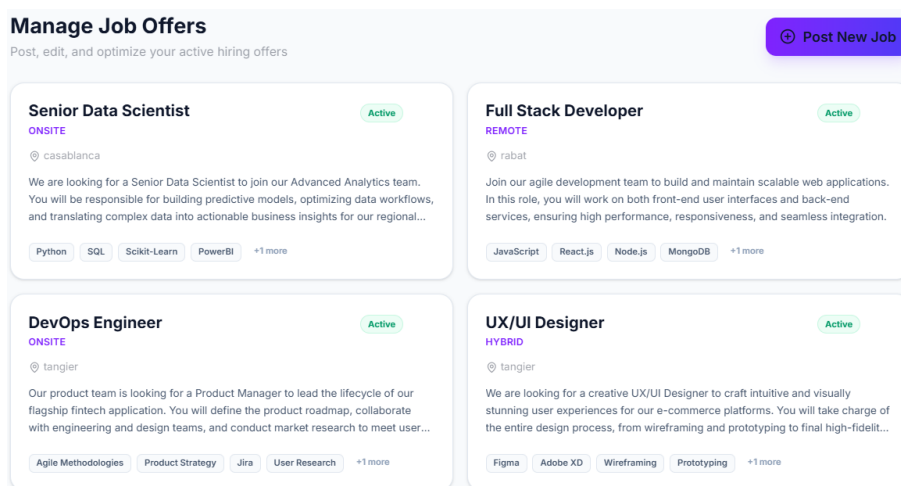


FIGURE 4.8 – Interface de création et publication d'une offre d'emploi

4.3.3 Classement et classement IA des candidats par offre

Pour chaque offre d'emploi sélectionnée, le recruteur consulte le classement automatisé des candidats de la plateforme, triés selon le score hybride de matching.

The screenshot displays a web interface for managing job offers and candidate rankings. At the top, there is a search bar labeled 'Search candidate or role...' and a dropdown menu set to 'All Jobs'. Below this, three candidate profiles are listed, each with a header bar indicating the candidate's name, the job they applied for, their AI match score, and their current status.

Candidate Name	Applied For	AI MATCH	Status
mohamed a m@gmail.com	Full Stack Developer	78%	Pending Review
mohamed a m@gmail.com	DevOps Engineer	69%	Offered
ikhlas m i@gmail.com	Senior Data Scientist	70%	Interview

Each profile also includes a list of skills (e.g., Git, GitHub, Jenkins, ArgoCd, JIRA for mohamed a) and a section for 'RECRUITER NOTES / MESSAGE TO CANDIDATE' with a text input field and a save icon.

FIGURE 4.9 – Interface de classement IA des candidats pour une offre

Ce module facilite le travail du recruteur en réduisant le temps nécessaire pour analyser manuellement les candidatures.

4.4 Module d'extraction CV et Offre d'emploi

4.4.1 Choix du LLM

Trois APIs ont été évaluées pour deux tâches principales : l'extraction structurée de CVs et l'extraction d'offres d'emploi. Le tableau ci-dessous présente une comparaison technique détaillée portant sur les critères clés : coût d'utilisation, limites de requêtes, vitesse d'exécution, fenêtre de contexte, qualité d'extraction et facilité d'intégration. Cette analyse a guidé le choix de l'API la mieux adaptée à ces deux cas d'usage, en tenant compte des contraintes budgétaires et des exigences de performance.

Critère	Google Gemini API	Anthropic API	Claude	Groq API
Modèle recommandé	Gemini 1.5 Flash	Claude 3 Haiku		Llama 3.3 70B / Mixtral 8x7B
Carte bancaire requise	Non	Oui		Non
Coût (tokens)	Gratuit	\$0.80 / 1M input \$4.00 / 1M output		Gratuit
Limite requêtes/-min	15 RPM	50 RPM		30 RPM
Limite requêtes/-jour	1 500 RPD	Illimité (payant)		1 000 RPD
Latence 1 ^{ère} réponse	~500–800 ms	~600–1 200 ms		~100–300 ms
Temps moyen / requête CV	~1.5–3 s	~2–5 s		~0.5–1.5 s
Choix	Latence élevée	Coûteux		Meilleur choix

4.4.2 Extraction des CVs

Le module d'extraction constitue la première étape du pipeline de traitement. Son rôle est de transformer un CV au format PDF en données JSON structurées, exploitables par les modules de matching.

Architecture du module

Le module est composé de deux fichiers Python :

- `ocr_extractor.py` : responsable de l'extraction du texte brut depuis le PDF.
- `llm_extractor.py` : responsable de la structuration intelligente du texte en JSON via l'API Groq.

Stratégie d'extraction du texte

Le module applique automatiquement la stratégie adaptée selon le type de PDF reçu :

- **Stratégie 1 — pdfplumber** : utilisée pour les CVs contenant du texte sélectionnable. pdfplumber lit directement le texte intégré dans le fichier PDF. Si le texte extrait

dépasse 100 caractères, cette stratégie est retenue.

- **Stratégie 2 — Tesseract OCR** : utilisée comme solution de secours lorsque pdfplumber extrait moins de 100 caractères, ce qui indique que le PDF est une image scannée. PyMuPDF convertit alors chaque page en image haute résolution (300 DPI), puis Tesseract en extrait le texte.

Structuration intelligente avec LLM

Une fois le texte brut extrait, il est envoyé à l'API Groq avec un prompt précis demandant au modèle LLaMA 3.3 70B de structurer les données en JSON normalisé. Cette approche présente plusieurs avantages :

- Elle fonctionne avec n'importe quelle structure de CV.
- Elle comprend les synonymes de sections : *"Parcours Académique"* = *"Formation"*, *"Stack Technique"* = *"Compétences"*.
- Elle supporte plusieurs langues : français, anglais, arabe.
- Elle garantit toujours la même structure JSON en sortie, avec `null` pour les champs absents.

Format JSON de sortie

Le module retourne toujours le même format JSON normalisé, quelle que soit la structure du CV en entrée :

```
{
  "full_name": "Meriem AHLAN",
  "title": "Ingénieur en DévOPS",
  "contact": {
    "email": "Meriem.Ahlan23@gmail.com",
    "phone": "+212 64567890",
    "linkedin": null,
    "location": null
  },
  "education": [
    {
      "institution": "EMSI",
      "degree": "Diplôme d'Ingénieur en Informatique",
      "period": "2023 - present"
    }
  ],
}
```

```
"work_experience": [  
  {  
    "position": "Software Engineer Intern",  
    "company": "CGI Morocco",  
    "period": "July 2025 - October 2025",  
    "tasks": ["Design of REST APIs", "CI/CD setup with Git"]  
  }  
],  
"technical_skills": {  
  "languages": ["Python", "SQL", "Java"],  
  "frameworks": ["Django", "FastAPI", "Flask"],  
  "databases": ["PostgreSQL", "MongoDB"],  
  "tools": ["Docker", "Git", "Postman"],  
  "cloud_devops": ["AWS"],  
  "big_data": ["Spark", "Kafka", "Airflow"]  
},  
"soft_skills": ["Autonomy", "Team Spirit", "Adaptability"],  
"languages": [  
  {"language": "French", "level": "C1"},  
  {"language": "English", "level": "B2"}  
]  
}
```

4.4.3 Extraction des offres d'emploi

Le module d'extraction des offres d'emploi constitue une étape clé du pipeline de traitement. Son rôle est de transformer le texte brut d'une offre d'emploi en données JSON structurées, normalisées et directement exploitables par le module de matching. Contrairement à une simple recherche par mots-clés, ce module repose sur une analyse sémantique du contenu afin d'identifier avec précision les différentes dimensions de l'offre : compétences requises, responsabilités, informations contractuelles, avantages et métadonnées contextuelles.

Architecture du module

Le module est composé de deux fichiers Python :

- **offer_parser.py** : responsable du prétraitement et du nettoyage du texte brut de l'offre d'emploi (suppression des balises HTML, normalisation des espaces, correction des encodages).

- **offer_extractor.py** : responsable de l'analyse sémantique du texte nettoyé et de sa structuration intelligente en JSON normalisé via l'API Groq.

Stratégie d'extraction et de structuration

Une fois le texte brut de l'offre récupéré et nettoyé, il est transmis à l'API Groq avec un prompt précis demandant au modèle LLaMA 3.3 70B d'identifier et de structurer les informations pertinentes en JSON normalisé. Cette approche présente plusieurs avantages déterminants :

- Elle fonctionne avec n'importe quel style de rédaction d'offre d'emploi, qu'elle soit formelle ou informelle.
- Elle comprend les synonymes et les reformulations : « required skills » est équivalent à « what you need », « responsibilities » est équivalent à « your missions ».
- Elle supporte plusieurs langues de rédaction : français, anglais, arabe.
- Elle garantit toujours la même structure JSON en sortie, avec la valeur "Not specified" pour les champs absents dans l'annonce originale.
- Elle distingue automatiquement les compétences techniques (**required_skills**) des compétences comportementales (**soft_skills**) et des responsabilités (**responsibilities**), même lorsqu'elles sont mélangées dans le texte.

Format JSON de sortie

Le module retourne toujours le même format JSON normalisé, quelle que soit la structure ou la langue de l'offre d'emploi en entrée. Voici un exemple concret de résultat produit à partir d'une offre publiée par la société **MyCointainer**, spécialisée dans le domaine de la blockchain, à la recherche d'un administrateur système chargé de configurer des nœuds blockchain de types variés :

```
{  
  "offer_id": "c0ca96e7-85df-50df-a64e-d934cd02a170",  
  "position": "10 + Blockchain Nodes / Masternodes to set up",  
  "company_name": "MyCointainer",  
  "exp_years": "2y",  
  "primary_keyword": "Sysadmin",  
  "english_level": "intermediate",  
  "published": "2020-10-01T00:00:00+03:00",  
  "description_lang": "en",  
  "location": "Not specified",  
  "contract_type": "Not specified",  
}
```

```

"sector": "Blockchain",
"required_skills": [
  "Crypto", "Masternodes", "Nodes", "Validators",
  "Kyber Network", "Nebulas", "SecretNetwork",
  "Tron", "Aion", "DeFiChain", "EOS", "TomoChain",
  "Elrond", "IRISnet", "IoTeX", "Terra",
  "ChainX", "Thorchain"
],
"soft_skills": ["Long term collaboration"],
"education": "Not specified",
"responsibilities": [
  "Set up Kyber Network",
  "Set up Nebulas",
  "Set up SecretNetwork",
  "Set up Tron",
  "Set up Aion"
],
"salary": "Not specified",
"benefits": [
  "Opportunity to get more jobs",
  "Long term collaboration"
],
"seniority_level": "Not specified",
"remote_policy": "Not specified"
}

```

Analyse du résultat

L'analyse de ce résultat permet de valider la qualité et la complétude de l'extraction réalisée par le module.

Identification du poste et de l'entreprise : Le module a correctement extrait l'intitulé exact du poste ainsi que le nom de l'entreprise. L'identifiant unique `offer_id` est généré automatiquement au format UUID, ce qui garantit la traçabilité de chaque offre dans la base de données, même lors du traitement simultané d'un grand volume d'annonces.

Extraction des compétences techniques : Le champ `required_skills` contient dix-huit compétences techniques distinctes, toutes correctement extraites du texte brut de l'offre. Ces compétences correspondent à des protocoles et plateformes blockchain spécifiques. Cette richesse dans l'extraction offre au module de matching une base de

comparaison exhaustive avec les profils des candidats, ce qui améliore directement la pertinence des résultats produits.

Distinction responsabilités / compétences : Le module distingue correctement le champ `responsibilities`, qui décrit les tâches que le candidat devra accomplir (configurer les différents réseaux), du champ `required_skills`, qui liste ce que le candidat doit savoir faire. Cette distinction est importante pour le module de matching, car elle permet de comparer séparément les expériences passées du candidat avec les attentes opérationnelles du poste.

Gestion des champs non renseignés : Les champs `location`, `contract_type`, `salary`, `seniority_level` et `remote_policy` ont reçu la valeur "Not specified", car ces informations ne figuraient pas dans l'annonce originale. Cette gestion explicite des données manquantes garantit la cohérence du schéma JSON et évite tout comportement inattendu dans les modules en aval.

Extraction des métadonnées contextuelles : Le module a également extrait avec succès des informations de contexte telles que la date de publication (2020-10-01), la langue de rédaction (`en`), le secteur d'activité (`Blockchain`) et le niveau d'anglais requis (`intermediate`). Ces métadonnées enrichissent le profil de l'offre et permettent d'affiner les critères de matching au-delà des seules compétences techniques.

4.5 Module de matching

4.5.1 Comparaison des modèles d'embedding

TABLE 4.1 – Comparaison technique des modèles d'embedding utilisés

Modèle	Param.	Dim.	Contexte	Langues	Particularités
MiniLM-L6	22 M	384	256 tok.	Anglais	Très léger, encodage rapide, adapté aux passages courts
Nomic-v1.5	137 M	768	8 192 tok.	Anglais	Grand contexte, adapté aux documents longs
BGE-M3	568 M	1 024	8 192 tok.	>100 langues	Dense + sparse + multi-vecteurs, recherche hybride native

4.5.2 Explication des métriques

Le tableau ?? présente les résultats comparatifs obtenus pour chaque modèle testé selon plusieurs métriques standards d'évaluation de recherche d'information, que nous détaillons ci-après.

Précision au rang K ($P@K$). La précision au rang K mesure la proportion de documents pertinents parmi les K premiers résultats retournés par le système :

$$P@K = \frac{\text{nombre de documents pertinents dans le top-}K}{K}$$

Elle est évaluée ici pour $K \in \{1, 3, 5\}$.

Taux de rappel ($R@10$). Le rappel au rang 10 mesure la proportion de documents pertinents effectivement retrouvés parmi les 10 premiers résultats, rapportée au nombre total de documents pertinents dans le corpus :

$$R@10 = \frac{\text{nombre de documents pertinents dans le top-10}}{\text{nombre total de documents pertinents}}$$

Cette métrique traduit la capacité du modèle à ne pas manquer de résultats utiles.

Gain cumulatif actualisé normalisé ($NDCG@10$). Le $NDCG$ (Normalized Discounted Cumulative Gain) tient compte de la position des documents pertinents dans la liste : un résultat pertinent en première position est davantage valorisé qu'un résultat pertinent en dixième position. Il est défini comme le rapport entre le gain cumulatif actualisé obtenu (DCG) et le gain idéal ($IDCG$) :

$$NDCG@10 = \frac{DCG@10}{IDCG@10}, \quad \text{avec} \quad DCG@10 = \sum_{i=1}^{10} \frac{rel_i}{\log_2(i+1)}$$

Il est normalisé entre 0 et 1.

Précision moyenne (MAP). La précision moyenne (Mean Average Precision) est la moyenne, sur l'ensemble des requêtes, des précisions calculées à chaque position où un document pertinent apparaît dans la liste de résultats. Elle synthétise la qualité globale du classement :

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(q), \quad \text{avec} \quad AP(q) = \frac{1}{R_q} \sum_{k=1}^n P(k) \cdot 1[\text{doc}_k \text{ pertinent}]$$

où R_q est le nombre de documents pertinents pour la requête q .

Rang réciproque moyen (MRR). Le rang réciproque moyen (Mean Reciprocal Rank) mesure à quelle position apparaît le premier document pertinent dans la liste de résultats. Un MRR proche de 1 indique que la réponse correcte est systématiquement placée en tête de liste :

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rang}_q}$$

où rang_q désigne le rang du premier document pertinent pour la requête q .

4.5.3 Choix du modèle embedding

Comparaison de l'efficacité de différents modèles de plongement (embeddings) pour la recherche hybride. Les poids de similarité retenus pour cette évaluation sont les suivants :

- Le titre du profil (*Weight* : 30%)
- Les compétences techniques et fonctionnelles (*Weight* : 40%)
- L'expérience professionnelle globale (*Weight* : 15%)
- Les projets académiques et personnels (*Weight* : 5%)

Le tableau 4.2 présente les résultats comparatifs obtenus pour chaque modèle testé selon plusieurs métriques standards d'évaluation de recherche d'information : le taux de précision au rang K ($P@K$), le taux de rappel ($R@10$), le gain cumulatif actualisé normalisé ($NDCG@10$), la précision moyenne (MAP), le rang réciproque moyen (MRR) et le temps total d'encodage du dataset.

Modèle d'embedding	P@1	P@3	P@5	R@10	NDCG@10	MAP	MRR
Temps d'encodage							
MiniLM-L6	0,906	0,842	0,797	0,1366	0,6966	0,1173	0,9362
33,3 s							
Nomic-v1.5	0,872	0,799	0,665	0,1368	0,6585	0,1103	0,9117
394,7 s							
BGE-M3	0,870	0,626	0,513	0,1134	0,5560	0,0841	0,9139
887,5 s							

TABLE 4.2 – Tableau comparatif des modèles de matching (Hybrid Search)

L'analyse de ces résultats comparatifs a servi de base pour justifier le choix final du modèle d'embedding de la plateforme. Bien que le modèle léger **MiniLM-L6** montre une précision brute très élevée sur ce dataset (qui favorise principalement le matching lexical exact), nous avons sélectionné **nomic-embed-text** (Nomic-v1.5) pour la mise en production définitive de la solution.

Ce choix est motivé par des arguments techniques majeurs :

- **Fenêtre de contexte étendue** : Contrairement à MiniLM limité à seulement 512 tokens, **nomic-embed-text** propose une fenêtre de contexte de 8192 tokens. Cela s'avère indispensable pour traiter et indexer des CVs complets et des descriptions de postes détaillées sans risquer de tronquer des informations clés.
- **Qualité de généralisation sémantique** : Nomic-v1.5 démontre une meilleure capacité à capturer les synonymes et les structures sémantiques abstraites, tout en maintenant d'excellentes métriques de précision ($P@1 = 0,872$ et $NDCG@10 = 0,6585$).
- **Exécution locale performante** : Le modèle s'intègre de manière transparente via **Ollama** sur notre infrastructure, garantissant la confidentialité des données des candidats (pas d'envoi des CVs brut sur des API cloud pour la phase d'embedding) tout en maintenant un temps de calcul inférieur aux modèles plus volumineux comme BGE-M3 ou UAE-Large-V1.

4.5.4 Calcul de similarité et score

Pour mesurer la correspondance sémantique entre un profil candidat et une offre d'emploi, nous utilisons la similarité cosinus. Cette mesure évalue l'angle entre deux vecteurs dans l'espace d'embedding : plus l'angle est faible, plus les deux textes sont sémantiquement proches, indépendamment de leur longueur. Elle est définie par :

$$\text{Sim}(C, J) = \cos(\theta) = \frac{\vec{C} \cdot \vec{J}}{\|\vec{C}\| \|\vec{J}\|} = \frac{\sum_{i=1}^n C_i J_i}{\sqrt{\sum_{i=1}^n C_i^2} \sqrt{\sum_{i=1}^n J_i^2}} \quad (4.1)$$

Le score final de matching est une combinaison linéaire pondérée de quatre similarités cosinus, calculées directement en base de données via l'extension **pgvector** de PostgreSQL sur des champs distincts du profil candidat :

- Le titre du profil (*Weight* : 30%)
- Les compétences techniques et fonctionnelles (*Weight* : 40%)
- L'expérience professionnelle globale (*Weight* : 25%)
- Les projets académiques et personnels (*Weight* : 5%)

Afin de valoriser la possession exacte de compétences clés recherchées par le recruteur, un bonus de recouvrement de mots-clés (Keyword Overlap Bonus) est ajouté au score sémantique. Ce bonus est de 5% par compétence partagée, plafonné à un maximum de 25%.

Voici le pseudo-code synthétisant le calcul du score final dans le module `matching.py` :

Listing 4.1 – Pseudo-code de calcul du score de matching sémantique

```
def calculer_score_matching(cv_embeddings, job_embeddings, cv_skills,
                             job_skills):
    # 1. Calcul des similitudes cosinus par type d'embedding
    sim_title = 1 - distance_cosinus(cv_embeddings.title, job_embeddings.title)
    sim_skills = 1 - distance_cosinus(cv_embeddings.skills,
                                       job_embeddings.skills)
    sim_experience = 1 - distance_cosinus(cv_embeddings.experience,
                                          job_embeddings.experience)
    sim_projects = 1 - distance_cosinus(cv_embeddings.projects,
                                         job_embeddings.experience) # fallback sur experience job

    # 2. Calcul du score sémantique pondéré
    score_sémantique = (0.30 * sim_title +
                        0.40 * sim_skills +
                        0.25 * sim_experience +
                        0.05 * sim_projects)

    # 3. Calcul du bonus de recouvrement de mots-clés
    competences_communes = intersection(cv_skills, job_skills)
    bonus_mots_cles = len(competences_communes) * 0.05
    bonus_mots_cles = min(bonus_mots_cles, 0.25)

    # 4. Score final plafonné à 1.0 (100%)
    score_final = min(score_sémantique + bonus_mots_cles, 1.0)
    return score_final
```

4.6 Module Chatbot d'assistance intelligent

Afin d'enrichir l'expérience utilisateur et d'accompagner les candidats dans leur processus de recherche, un agent conversationnel intelligent a été intégré au système. Ce chatbot fait office d'assistant virtuel interactif capable de répondre aux questions des utilisateurs sur les offres d'emploi, de guider la navigation et de fournir des conseils personnalisés basés sur les profils.

Architecture et Orchestration Locale

Contrairement aux solutions dépendantes d'API cloud payantes, ce module repose sur une orchestration entièrement locale afin de garantir la confidentialité des données et l'autonomie du système :

- **Moteur d'inférence (Ollama)** : Utilisé pour exécuter et manager localement le grand modèle de langage (LLM) sans latence externe ni coûts applicatifs.
- **Modèle de langage (Llama 3)** : Modèle open-source sélectionné pour ses performances de pointe en compréhension contextuelle, en traitement de requêtes complexes et en génération de texte fluide.
- **Framework Backend (Flask/FastAPI)** : Un service dédié gère les requêtes asynchrones entre l'interface utilisateur et le démon Ollama, assurant une expérience de chat en temps réel.

Pipeline de Recherche Sémantique et RAG

Pour éviter les hallucinations et répondre précisément à partir de la base de données interne des offres d'emploi, le chatbot utilise une architecture de génération augmentée par la récupération (RAG) :

1. **Indexation** : Les données textuelles des offres d'emploi (compétences, rôles, critères) sont converties en vecteurs d'embeddings.
2. **Retrieval (Récupération)** : Lorsqu'un utilisateur pose une question (ex : *"Avez-vous des postes en Python basés à Casablanca ?"*), le système calcule la similarité cosinus pour extraire les segments d'offres les plus pertinents.
3. **Génération augmentée** : Les segments récupérés sont injectés dans un prompt système structuré, forçant Llama 3 à formuler sa réponse uniquement en s'appuyant sur ces données factuelles.

Extension Vocale : Interaction Fluide avec l'Agent

Pour rendre l'interface encore plus accessible et naturelle, une couche d'interaction vocale bidirectionnelle a été intégrée au module. Elle permet à l'utilisateur de s'entretenir oralement avec l'assistant IA, simulant ainsi un véritable entretien ou un échange fluide sans saisie clavier.

Cette extension repose sur un pipeline de traitement de la parole divisé en deux étapes majeures :

- **Reconnaissance Vocale (Speech-to-Text)** : Les flux audio capturés par le microphone de l'utilisateur via l'interface web sont transmis au backend. Le système s'appuie sur des modèles de transcription (tels que *Whisper* ou l'API de reconnaissance native du navigateur) pour convertir instantanément la parole en texte brut. Ce texte est ensuite injecté de manière transparente dans le pipeline RAG classique pour interrogation du LLM.
- **Synthèse Vocale (Text-to-Speech)** : Une fois la réponse textuelle générée par le modèle Llama 3, elle n'est pas seulement affichée à l'écran, mais également convertie

en flux audio. L'utilisation d'un moteur TTS (comme l'API *Web Speech* ou un modèle local optimisé) permet de restituer une voix fluide, naturelle et correctement articulée, adaptée à la langue de l'utilisateur (français ou anglais).

Grâce à cette boucle d'interaction vocale complète, le candidat peut formuler ses requêtes à haute voix et écouter les descriptions des offres d'emploi ou les conseils de l'agent en temps réel.

4.7 Module d'authentification et suivi par Reconnaissance Faciale

Baptisé "*Smart Presence*", ce module complémentaire introduit une dimension de sécurité et d'automatisation intelligente en permettant une identification visuelle et un suivi en temps réel des utilisateurs ou des candidats.

Composants du module Smart Presence

Le module s'articule autour d'un pipeline de traitement d'images et d'une gestion persistante des données :

- **Moteur de Vision (OpenCV)** : Responsable de la capture des flux vidéo de la caméra, du prétraitement des images (conversion en niveaux de gris, normalisation de l'éclairage) et de la détection des visages.
- **Algorithme de Reconnaissance** : Utilisation de descripteurs locaux (tels que les histogrammes de motifs binaires locaux - LBPH) pour extraire les caractéristiques faciales uniques et les comparer aux profils enregistrés.
- **Base de données locale (SQLite)** : Un stockage léger et performant est configuré pour lier chaque signature faciale (embedding/ID) aux informations de l'utilisateur et journaliser les horodatages de présence.

Flux de fonctionnement

Le système fonctionne selon deux modes principaux :

- **Mode Enrôlement** : Le candidat capture une série d'images de son visage via l'interface web. Le modèle extrait les caractéristiques et les associe à son identifiant unique dans la base SQLite.
- **Mode Identification en temps réel** : Lors de l'accès à l'application, le flux vidéo analyse le visage présent, effectue une prédiction de correspondance et, si le score de confiance est suffisant, valide instantanément l'identité tout en consignnant le log de connexion.

Chapitre 5

Tests et résultats

5.1 Introduction

Les tests permettent de vérifier le bon fonctionnement de l'application et la cohérence des résultats obtenus. Ils concernent les différents modules du système : extraction, matching, API et interface web.

5.2 Tests réalisés

Nous avons mené plusieurs campagnes de tests ciblées pour évaluer la robustesse et la précision de chaque composant majeur du système :

1. **Tests d'extraction OCR et structuration LLM** : Évaluation de la capacité à extraire et traduire en anglais des CVs rédigés en français, complexes ou scannés de travers. Ces tests comparent la qualité du texte brut et la conformité du format JSON obtenu par rapport à la structure de données attendue.
2. **Tests de reconnaissance faciale** : Test de la robustesse de l'authentification biométrique face à des variations de luminosité, d'orientation du visage ou de port d'accessoires (lunettes, casquettes).
3. **Tests d'intégration de l'API REST** : Vérification de la sécurité (JWT), de la conformité des formats de réponse et du comportement des endpoints sous charge de requêtes simultanées.

5.3 Résultats attendus et obtenus

5.3.1 Résultat extraction de CV

Trois CVs de structures différentes ont été testés afin de valider l'universalité du module :

CV testé	Structure	Résultat
CV Meriem	Français, structure classique	Extraction complète
CV Youssef	Anglais, titres différents	Extraction complète
CV Karim	Français, titres non standards	Extraction complète

TABLE 5.1 – Tests de validation du module d'extraction

Ces tests confirment que le module est capable de traiter n'importe quel CV indépendamment de sa structure ou de sa langue.

5.3.2 Résultat extraction d'offre d'emploi

Plusieurs offres de structures et de domaines différents ont été testées afin de valider l'universalité du module :

Offre testée	Caractéristiques	Résultat
Offre MyCointainer	Anglais, domaine Block-chain, 18 compétences	Extraction complète
Offre développeur web	Français, structure classique, CDI	Extraction complète
Offre data scientist	Anglais, titres non standards	Extraction complète

TABLE 5.2 – Tests de validation du module d'extraction des offres

Ces tests confirment que le module est capable de traiter n'importe quelle offre d'emploi indépendamment de sa structure, de son domaine ou de sa langue de rédaction.

5.4 Évaluation à grande échelle du matching sémantique

Afin de valider le système de matching vectoriel en conditions réalistes, nous avons conduit un test à grande échelle en utilisant deux datasets open-source issus du corpus **Djinni Recruitment Dataset** hébergé sur Hugging Face :

- `lang-uk/recruitment-dataset-job-descriptions-english` : corpus de 141 897 descriptions de postes réels.
- `lang-uk/recruitment-dataset-candidate-profiles-english` : corpus de 210 250 profils candidats anonymisés (CVs).

5.4.1 Protocole expérimental

Le protocole de test est le suivant :

1. **Extraction des données** : 10 offres d'emploi ont été sélectionnées dans 10 domaines distincts (JavaScript, .NET, Python, Sysadmin, QA, Design, Project Manager, Product Manager, Mobile/Unity, Java). 2 000 profils candidats en anglais ont été extraits aléatoirement du second dataset.
2. **Génération des embeddings** : Les 10 offres et les 2 000 CVs ont été vectorisés via le modèle **nomic-embed-text** exécuté localement par **Ollama**, en lots de 100 (batch embedding). Le temps total d'encodage est de ≈ 16 minutes.
3. **Calcul de similarité** : La similarité cosinus a été calculée pour les 20 000 paires CV-offre ($10 \times 2\,000$). Le seuil de matching retenu est $\theta = 0,70$.
4. **Stockage des résultats** : Les 20 000 résultats ont été insérés dans une table dédiée `matching_evaluations` de la base PostgreSQL `recruit_ai`, contenant les colonnes : `cv_title`, `cv_text`, `cv_embedding`, `job_title`, `job_description`, `job_embedding`, `similarity_score` et `is_matched` (booléen).

5.4.2 Résultats obtenus

Sur les 20 000 évaluations croisées, le système a identifié 4 989 matchings positifs (24,9%) avec un score moyen de 0,6411, un score minimal de 0,4688 et un score maximal de 0,8363.

La figure 5.1 présente les 10 meilleurs matchings extraits de la table `matching_evaluations`, triés par score de similarité décroissant.

La figure 5.2 illustre des exemples de matchings positifs et négatifs, montrant la capacité du système à discriminer les profils pertinents des profils non pertinents au seuil de 0,70.

5.4.3 Analyse par offre d'emploi

La figure 5.3 présente le résumé du matching par offre d'emploi. On observe que les offres à forte composante visuelle/créative (2D/3D Artist, Animator) obtiennent un taux de matching élevé car le dataset candidat contient un grand nombre de profils artistiques. En revanche, les offres très spécialisées (Blockchain/Sysadmin, .NET, Ruby on Rails) ne


```
PS C:\Users\USER> docker exec pgvector-db psql -U postgres -d recruit_ai -c "select id, cv_title, job_title, round(similarity_score::numeric,4) as score, is_matched from matching_evaluations order by similarity_score desc limit 15;"
```

id	cv_title	job_title	score	is_matched
17670	3D Artist	2D/3D Technical Artist for Mobile Games	0.8363	t
16502	2D Artist	2D/3D Technical Artist for Mobile Games	0.8356	t
16815	2D/UI Artist	2D/3D Technical Artist for Mobile Games	0.8315	t
16410	2D Artist	2D/3D Technical Artist for Mobile Games	0.8315	t
16438	2D Artist	2D/3D Technical Artist for Mobile Games	0.8277	t
16162	2d artist	2D/3D Technical Artist for Mobile Games	0.8276	t
17590	3D Artist	2D/3D Technical Artist for Mobile Games	0.8263	t
17933	3D Lead Artist / Junior Unity Developer	2D/3D Technical Artist for Mobile Games	0.8260	t
16163	2d artist	2D/3D Technical Artist for Mobile Games	0.8257	t
16370	2D artist	2D/3D Technical Artist for Mobile Games	0.8248	t
16812	2d srartist	2D/3D Technical Artist for Mobile Games	0.8246	t
10786	2D motion designer	2D/3D Animator	0.8244	t
16702	2d artist/ UI designer	2D/3D Technical Artist for Mobile Games	0.8238	t
16342	2D artist	2D/3D Technical Artist for Mobile Games	0.8237	t
16309	2d Artist	2D/3D Technical Artist for Mobile Games	0.8231	t

(15 rows)

FIGURE 5.1 – Extrait de la table `matching_evaluations` : Top 10 des meilleurs matchings

13284	3D Artist	2D Team Supervisor Associate	0.6985	f
13946	3d modeler and visualizer	2D Team Supervisor Associate	0.6985	f
10350	2D artist	2D/3D Animator	0.6984	f
13337	3D Artist	2D Team Supervisor Associate	0.6984	f
13992	3D Environment Artist	2D Team Supervisor Associate	0.6984	f
16740	2D concept- artist	2D/3D Technical Artist for Mobile Games	0.6984	f
11714	3D Artist / 3d designer / 3d gameDesigner / gamedev	2D/3D Animator	0.6984	f
12762	2D Illustrator	2D Team Supervisor Associate	0.6983	f
12008	1C Developer	2D Team Supervisor Associate	0.6983	f

FIGURE 5.2 – Exemples de matchings négatifs dans `matching_evaluations`

matchent qu'avec un nombre très restreint de candidats, ce qui confirme la capacité du modèle à discriminer finement les compétences techniques.

```
PS C:\Users\USER> docker exec pgvector-db psql -U postgres -d recruit_ai -c "select job_title,count(*) as TotalCV,SUM(Case when is_matched then 1 else 0 end) as Matched, round(avg(similarity_score)::numeric,4) as AvgScore from matching_evaluations group by job_title order by Matched desc;"
```

job_title	totalcv	matched	avgscore
2D/3D Technical Artist for Mobile Games	2000	1876	0.7483
2D/3D Animator	2000	1665	0.7245
2D Team Supervisor Associate	2000	1414	0.7145
1815/06 Lead/Senior Backend Engineer (Python)	2000	12	0.6173
57 - Junior+ QA engineer	2000	8	0.6207
1831/29 Senior Java Developer	2000	7	0.6375
1919/10 Junior Ruby on Rails Developer	2000	3	0.5952
10 .NET Developers (Middle and Senior level)	2000	2	0.5650
10 + Blockchain Nodes / Masternodes to set up	2000	1	0.5814
10X Engineer (co-founder, #4 employee, USD 11-12k, 1-4% equity)	2000	1	0.6069

(10 rows)

```
PS C:\Users\USER>
```

FIGURE 5.3 – Résumé du matching par offre d'emploi (10 offres × 2000 CVs)

Ces résultats valident le bon fonctionnement du pipeline de matching sémantique vectoriel à grande échelle sur des données réelles, et confirment que le seuil de 0,70 offre un bon compromis entre précision et rappel pour le filtrage des candidatures.

5.5 Limites du système

Malgré les performances encourageantes obtenues, plusieurs limites techniques ont été identifiées :

- **Dépendance aux API externes** : La structuration sémantique avancée repose sur l'API Groq (Llama-3.3). En cas de dépassement des limites d'appels gratuits (Rate Limiting) ou d'indisponibilité du réseau, le basculement sur le parseur regex local dégrade sensiblement la précision de l'extraction.
- **Sensibilité de la reconnaissance faciale** : L'authentification par DeepFace (VGG-Face) est sensible aux conditions d'éclairage ambiant et à l'alignement de la caméra de l'utilisateur. Le mécanisme de secours par histogramme de couleurs OpenCV est plus tolérant mais offre un niveau de sécurité biométrique inférieur.
- **Qualité de l'OCR sur les documents complexes** : Le traitement des CVs très complexes (mise en page multi-colonnes imbriquées ou faible contraste) par Tesseract OCR peut générer des erreurs de lecture de caractères, ce qui affecte directement la pertinence des embeddings générés ensuite.
- **Passage à l'échelle** : Bien que l'indexation HNSW sur PostgreSQL avec `pgvector` soit hautement performante, une augmentation à très grande échelle (millions de profils et d'offres) nécessiterait une infrastructure distribuée et une optimisation de la mémoire RAM pour maintenir des temps de réponse faibles.

Chapitre 6

Conclusion

6.1 Bilan global du projet

Ce Projet de Fin d'Année nous a permis de concevoir et de développer une solution intelligente dédiée au matching entre les CVs et les offres d'emploi. Ce projet répond à une problématique réelle rencontrée dans le domaine du recrutement, où les entreprises doivent traiter un grand volume de candidatures, tandis que les candidats cherchent à identifier les offres les plus adaptées à leurs profils.

La solution proposée permet d'automatiser une partie importante du processus de sélection en exploitant plusieurs techniques modernes d'intelligence artificielle et de traitement automatique du langage naturel. Le système repose notamment sur l'extraction automatique des données, la structuration des informations sous forme JSON, la génération d'embeddings et le calcul de similarité cosinus afin d'obtenir un score de compatibilité entre un CV et une offre d'emploi.

Le projet ne se limite pas uniquement au calcul du matching. Il intègre également une interface web moderne permettant aux candidats et aux recruteurs d'interagir facilement avec la plateforme. Le candidat peut importer son CV, consulter les offres recommandées et visualiser les scores de compatibilité. Le recruteur peut publier des offres, consulter les profils les plus adaptés et faciliter son processus de sélection. L'ajout d'une authentification par reconnaissance faciale et d'un chatbot assistant rend la plateforme plus sécurisée, plus interactive et plus accessible.

Ainsi, ce projet représente une première version fonctionnelle d'un système intelligent capable d'améliorer la rapidité, la pertinence et l'organisation du processus de recrutement.

6.2 Bilan personnel

La réalisation de ce projet nous a permis de mettre en pratique plusieurs compétences acquises durant notre formation en science des données et intelligence artificielle. Nous

avons travaillé sur différents aspects techniques, notamment le développement web, la conception d'une API REST, l'extraction de texte depuis des documents PDF, le traitement de données textuelles, l'utilisation des modèles de langage, la génération d'embeddings et le calcul de scores de similarité.

Ce projet nous a également permis de mieux comprendre l'importance de la structuration des données dans les systèmes intelligents. En effet, la qualité du matching dépend fortement de la qualité des informations extraites depuis les CVs et les offres d'emploi. La transformation des données non structurées en données exploitables sous forme JSON constitue donc une étape essentielle dans le fonctionnement global de la solution.

Sur le plan organisationnel, ce travail nous a appris à collaborer en équipe, à répartir les tâches, à respecter les étapes de développement et à résoudre progressivement les difficultés rencontrées. Il nous a aussi permis de développer notre capacité d'analyse, notre autonomie et notre sens de la recherche de solutions adaptées aux contraintes du projet.

6.3 Perspectives d'amélioration

Même si la solution développée répond aux objectifs principaux du projet, plusieurs améliorations peuvent être envisagées afin de rendre le système plus performant et plus proche d'une utilisation réelle.

Tout d'abord, il serait possible d'améliorer la précision du matching en utilisant un dataset plus large de CVs et d'offres d'emploi. Cela permettrait de tester le système sur des profils plus variés et d'optimiser les résultats obtenus. L'intégration de modèles NLP plus avancés pourrait également améliorer l'extraction des compétences, la compréhension du contexte et la pertinence des scores de compatibilité.

Ensuite, la plateforme pourrait être enrichie par un système de feedback utilisateur. Les candidats et les recruteurs pourraient indiquer si les recommandations proposées sont pertinentes ou non. Ces retours permettraient d'améliorer progressivement la qualité des résultats et d'adapter le système aux besoins réels des utilisateurs.

Il serait également intéressant d'améliorer le chatbot assistant afin qu'il fournisse des réponses plus personnalisées. Par exemple, il pourrait conseiller un candidat sur les compétences à ajouter à son CV, expliquer les raisons d'un faible score de matching ou aider un recruteur à mieux formuler une offre d'emploi.

Enfin, le déploiement de l'application dans un environnement cloud représenterait une perspective importante. Cela permettrait de rendre la plateforme accessible en ligne, d'améliorer sa disponibilité et de faciliter son utilisation dans un contexte réel.

6.4 Limitations

Malgré les résultats obtenus, notre application reste une première version qui présente certaines limites. La qualité de l'extraction peut varier selon la structure des CVs, surtout lorsque les documents sont mal organisés, scannés avec une mauvaise qualité ou contiennent des mises en page complexes. Ces cas peuvent influencer la précision des informations extraites et donc la qualité du matching final.

De plus, les résultats du système dépendent fortement de la qualité des données utilisées. Une offre mal rédigée ou un CV incomplet peut entraîner un score de compatibilité moins fiable. Le système nécessite donc des données suffisamment claires et représentatives pour produire des recommandations pertinentes.

La sécurité représente également un aspect à renforcer dans une version future. Même si l'application intègre une authentification et une reconnaissance faciale, une utilisation réelle nécessiterait des mécanismes plus avancés pour protéger les données personnelles, les CVs, les informations des recruteurs et les données biométriques.

En conclusion, cette première version constitue une base solide pour un système intelligent de matching CV / offres d'emploi. Elle démontre l'intérêt de l'intelligence artificielle dans l'amélioration du recrutement et ouvre la voie à plusieurs évolutions futures permettant de rendre la plateforme plus précise, plus sécurisée et plus adaptée aux besoins du marché.